Java Interface — 15 Codes

Beginner to Intermediate • Practice & Interview Revision

Core syntax to remember: interface → implements → interface reference → extends → multiple interfaces

1. Simple Interface

Basic interface and implements.

```
interface Student {

    void study();
}

class CollegeStudent implements Student {

    public void study() {
        System.out.println("Student is studying");
    }
}

public class Main {

    public static void main(String[] args) {

        CollegeStudent s = new CollegeStudent();

        s.study();
    }
}
```

2. Interface Reference + Class Object

An interface reference can point to an implementing-class object.

```
interface Student {

    void study();
}

class CollegeStudent implements Student {

    public void study() {
        System.out.println("Studying Java");
    }
}

public class Main {

    public static void main(String[] args) {

        Student s = new CollegeStudent();

        s.study();
    }
}
```

3. Interface With Multiple Methods

A class implementing an interface must implement its abstract methods.

```
interface Calculator {

    void add();

    void subtract();
```

```
    void multiply();
}

class MyCalculator implements Calculator {

    public void add() {
        System.out.println(10 + 5);
    }

    public void subtract() {
        System.out.println(10 - 5);
    }

    public void multiply() {
        System.out.println(10 * 5);
    }
}

public class Main {

    public static void main(String[] args) {

        MyCalculator c = new MyCalculator();

        c.add();
        c.subtract();
        c.multiply();
    }
}
```

4. Interface With Variables

Interface fields are implicitly public, static and final.

```
interface College {

    int FEES = 50000;

    void admission();
}

class Student implements College {

    public void admission() {

        System.out.println("College fees: " + FEES);
    }
}

public class Main {

    public static void main(String[] args) {

        Student s = new Student();

        s.admission();
    }
}
```

5. Interface + Constructor

Interfaces do not have constructors; implementing classes can have constructors.

```
interface Employee {

    void work();
}

class Developer implements Employee {
```

```
    Developer() {

        System.out.println("Developer object created");
    }

    public void work() {

        System.out.println("Developer writes code");
    }
}

public class Main {

    public static void main(String[] args) {

        Developer d = new Developer();

        d.work();
    }
}
```

6. Multiple Interfaces

A class can implement multiple interfaces.

```
interface Camera {

    void takePhoto();
}

interface MusicPlayer {

    void playMusic();
}

class Smartphone implements Camera, MusicPlayer {

    public void takePhoto() {

        System.out.println("Taking photo");
    }

    public void playMusic() {

        System.out.println("Playing music");
    }
}

public class Main {

    public static void main(String[] args) {

        Smartphone phone = new Smartphone();

        phone.takePhoto();
        phone.playMusic();
    }
}
```

7. Multiple Interfaces + Interface References

Each interface reference exposes the methods declared by its interface.

```
interface Camera {

    void click();
}

interface GPS {

    void location();
}

class Phone implements Camera, GPS {

    public void click() {

        System.out.println("Photo clicked");
    }

    public void location() {

        System.out.println("Location detected");
    }
}

public class Main {

    public static void main(String[] args) {

        Camera c = new Phone();
```

```
        c.click();

        GPS g = new Phone();

        g.location();
    }
}
```

8. Same Interface, Different Classes

Demonstrates runtime polymorphism.

```
interface Payment {

    void pay();
}

class UPI implements Payment {

    public void pay() {

        System.out.println("Payment using UPI");
    }
}

class CreditCard implements Payment {

    public void pay() {

        System.out.println("Payment using Credit Card");
    }
}

class Cash implements Payment {

    public void pay() {

        System.out.println("Payment using Cash");
    }
}

public class Main {

    public static void main(String[] args) {

        Payment p;

        p = new UPI();
        p.pay();

        p = new CreditCard();
        p.pay();

        p = new Cash();
        p.pay();
    }
}
```

9. Interface as a Method Parameter

A method parameter of interface type can accept any implementing object.

```
interface Payment {

    void pay();
}

class UPI implements Payment {

    public void pay() {
```

```
        System.out.println("Paid using UPI");
    }
}

class CreditCard implements Payment {

    public void pay() {

        System.out.println("Paid using Credit Card");
    }
}

public class Main {

    static void makePayment(Payment p) {

        p.pay();
    }

    public static void main(String[] args) {

        UPI u = new UPI();

        makePayment(u);

        CreditCard c = new CreditCard();

        makePayment(c);
    }
}
```

10. Interface + Array

An interface array can store objects of different implementing classes.

```
interface Payment {

    void pay();
}

class UPI implements Payment {

    public void pay() {

        System.out.println("UPI payment");
    }
}

class Card implements Payment {

    public void pay() {

        System.out.println("Card payment");
    }
}

class Cash implements Payment {

    public void pay() {

        System.out.println("Cash payment");
    }
}

public class Main {

    public static void main(String[] args) {

        Payment[] payments = {

            new UPI(),
```

```
            new Card(),
            new Cash()
        };

        for (Payment p : payments) {

            p.pay();
        }
    }
}
```

11. Default Method in Interface

Java 8+ interfaces can contain default methods with a body.

```
interface Printer {

    void print();

    default void start() {

        System.out.println("Printer started");
    }
}

class HPPrinter implements Printer {

    public void print() {

        System.out.println("Printing document");
    }
}

public class Main {

    public static void main(String[] args) {

        HPPrinter p = new HPPrinter();

        p.start();
        p.print();
    }
}
```

12. Overriding a Default Method

A class can override a default interface method.

```
interface Printer {

    default void start() {

        System.out.println("Normal printer started");
    }
}

class LaserPrinter implements Printer {

    @Override
    public void start() {

        System.out.println("Laser printer started");
    }
}

public class Main {

    public static void main(String[] args) {

        LaserPrinter p = new LaserPrinter();

        p.start();
    }
}
```

13. Static Method in Interface

Interface static methods are called using the interface name.

```
interface Calculator {
```

```
    static void showMessage() {

        System.out.println("Calculator interface");
    }

    int add(int a, int b);
}

class MyCalculator implements Calculator {

    public int add(int a, int b) {

        return a + b;
    }
}

public class Main {

    public static void main(String[] args) {

        Calculator.showMessage();

        MyCalculator c = new MyCalculator();

        System.out.println(c.add(10, 20));
    }
}
```

14. Interface Extending Another Interface

An interface can extend another interface.

```
interface BasicComputer {

    void turnOn();
}

interface AdvancedComputer extends BasicComputer {

    void connectInternet();
}

class Laptop implements AdvancedComputer {

    public void turnOn() {

        System.out.println("Laptop turned on");
    }

    public void connectInternet() {

        System.out.println("Internet connected");
    }
}

public class Main {

    public static void main(String[] args) {

        Laptop l = new Laptop();

        l.turnOn();
        l.connectInternet();
    }
}
```

15. Multiple Interface Inheritance

Shows an interface extending multiple interfaces and a class implementing the child interface.

```
interface Camera {

    void takePhoto();
}

interface Music {

    void playMusic();
}

interface SmartDevice extends Camera, Music {

    void connectWifi();
}

class Smartphone implements SmartDevice {

    public void takePhoto() {

        System.out.println("Taking photo");
    }

    public void playMusic() {

        System.out.println("Playing music");
    }

    public void connectWifi() {

        System.out.println("WiFi connected");
    }
}

public class Main {

    public static void main(String[] args) {

        SmartDevice phone = new Smartphone();

        phone.takePhoto();
        phone.playMusic();
        phone.connectWifi();
    }
}
```

Quick Revision

1. interface A { } **2.** class B implements A { } **3.** A obj = new B();

4. interface B extends A { } **5.** class C implements A, B { }

Key point: You normally cannot create an object directly from an interface. Instead, an interface reference can refer to an object of a class that implements the interface.